

Tema 1 — Modelo de cómputo, exclusión mutua y modelo de memoria

Fuentes: Teórica 01-Mutex, Práctica clase-01, Guía 1. **Ejercicio 1 del parcial.**

1. Cómo modelamos un programa concurrente

- **Programa secuencial:** una función de estado a estado. Mismo input, mismo output (determinismo).
- **Programa concurrente:** varios threads cuyas instrucciones se **entrelazan** (*interleaving*) en cualquier orden que respete el orden **interno** de cada thread.
 - El resultado **no es determinístico**: la misma entrada puede dar salidas distintas.
- **Instrucción atómica:** se ejecuta "de un saque", nadie se mete en el medio.
 - Por defecto asumimos que son atómicas **las lecturas y escrituras de variables** y las cuentas con variables locales.
 - **$x = x + 1$ NO es atómica.** Son 3 pasos:

```
tmp = x      // leo la compartida
tmp = tmp + 1 // cuenta local
x = tmp      // escribo la compartida
```

Si dos threads leen $x = 0$ antes de que alguno escriba, los dos escriben 1: **se perdió un incremento**. Esto es el ejemplo base de *race condition*.

- **Independencia del tiempo:** jamás podés razonar "este thread es más rápido" o "`sleep(1000)` asegura que el otro termina antes". El scheduler puede cortar en cualquier instrucción.

Estado, traza, grafo

- **Estado:** el valor de las variables + en qué instrucción está cada thread (su "program counter", PC).
- **Traza:** una ejecución concreta, un camino en el grafo de estados. En el parcial se escribe como tabla:

T1	T2	Estado
tmp = x		T1.tmp = 0
	tmp = x	T2.tmp = 0
x = tmp + 1		x = 1
	x = tmp + 1	x = 1 ← se perdió uno

- **Diagrama de transición de estados:** todos los estados posibles y cómo se pasa de uno a otro. Crece exponencialmente, así que solo sirve para programas chiquitos (Guía 1, ej 1–4).

2. Qué significa "correcto"

- **Safety** = algo malo **nunca** pasa (vale en *todo* estado). Ejemplo: "nunca hay dos en la sección crítica".
- **Liveness** = algo bueno **tarde o temprano** pasa. Ejemplo: "si quiero entrar, en algún momento entro".

- Un programa que no hace nada es *safe* pero inútil. Lo difícil es conseguir las dos cosas.
- **Fairness (débil):** asumimos que toda instrucción que queda **continuamente habilitada** termina ejecutándose. Sirve para descartar trazas absurdas donde un thread nunca corre.
 - **Importante:** cuando mostrás inanición, tu traza infinita tiene que ser *fair*.

3. El problema de la exclusión mutua

Cada thread repite: sección no crítica (SNC) → pre-protocolo → SECCIÓN CRÍTICA (SC) → post-protocolo.

Supuestos:

- La **SC siempre termina**.
- La **SNC puede no terminar** (un thread se puede quedar ahí para siempre).
- No se comparten variables entre la SC y la SNC.

Hay que garantizar **tres cosas**:

1. **Exclusión mutua** (safety): nunca hay 2 en la SC.
2. **Ausencia de deadlock** (liveness): si varios quieren entrar, **alguno** entra.
3. **Ausencia de inanición** (liveness): **todo** el que quiere entrar, entra.

En criollo: es un baño con una sola llave. (1) Nunca hay dos adentro. (2) Si hay cola, alguien entra. (3) Nadie se queda en la cola para siempre.

4. Los intentos de la teórica (solo con leer y escribir)

Algoritmo	Idea	Mutex	Deadlock-free	Starvation-free	Por qué falla
I: "levanto la bandera"	<code>while(flag); flag = true</code>	X	✓	X	los dos ven <code>flag = false</code> y entran antes de que alguno la levante
II: "levanto mi bandera"	<code>flag[yo] = true;</code> <code>while(flag[otro])</code>	✓	X	X	los dos levantan la bandera y se esperan para siempre (livelock)
III: turno	<code>while(turno != yo); ...;</code> <code>turno = otro</code>	✓	✓	X	si el otro se queda en la SNC, nunca me devuelve el turno
Dekker	II + III	✓	✓	✓	(solo para 2 threads)
Peterson	<code>flag[yo] = true; turno =</code> <code>otro; while(flag[otro] &&</code> <code>turno == otro)</code>	✓	✓	✓	(solo para 2 threads)
Bakery (panadería)	sacás número = 1 + máximo; entra el menor; empate → menor id	✓	✓	✓	(N threads)

Peterson, en criollo: "quiero entrar (flag), pero te cedo el paso (turno = otro)". Si los dos quieren a la vez, el último que cedió es el que espera.

Bakery: como en la panadería, sacás un número y atienden en orden. Como dos pueden sacar el mismo número, se desempata por id. Y mientras alguien está sacando número (`entrando[j]`), lo esperás.

Con instrucciones atómicas de hardware (más fácil)

- **test-and-set / exchange:** leo el valor viejo y escribo 1, todo junto. `repeat TAS(lock) until viejo == 0`.
- **compare-and-swap (CAS):** "si vale lo que espero, ponele el valor nuevo" (atómico). Es la base de todo lo lock-free del tema 4.
- **fetch-and-add:** `ticket. miTurno = FAA(ticket, 1); while(turno != miTurno); SC; FAA(turno, 1)`.
 - Guía 1 ej 14: al salir se hace `FAA(ticket, -1)`, o sea se **decrementa el ticket** en vez de avanzar el turno. ¿Qué rompe eso?

Todas estas soluciones hacen **busy-waiting** (gastan CPU girando en un `while`). Los semáforos (tema 2) arreglan eso.

5. Modelo de memoria (práctica 1, segunda mitad) — va en el inciso (c) del parcial

- Todo lo anterior supone **Consistencia Secuencial (SC)**: las instrucciones de todos se ejecutan en *algún* orden total que respeta el orden de cada thread.
- **El hardware real no cumple SC.**
 - **Store buffer:** cada core escribe primero en una colita privada y sigue de largo. Los demás cores todavía no ven esa escritura.
 - En **x86 (TSO)** esto permite que un *load* posterior "se adelante" a un *store* anterior (reordenamiento Store→Load).
 - **Dekker litmus:** `T1: x = 1; r1 = y, T2: y = 1; r2 = x`. Bajo SC es imposible que `r1 == r2 == 0`, pero en x86 real **pasa**.
 - **ARM/POWER** reordenan todavía más cosas.
- **FENCE** (barrera de memoria, `mfence`): "no sigas hasta vaciar el store buffer". Se pone entre el store y el load.
- **Instrucciones RMW atómicas** (`xchg`, `lock cmpxchg`, `lock xadd`): además de ser atómicas, en x86 ordenan la memoria.
- **El compilador también reordena**, o te guarda una variable en un registro. Un `while(!flag);` puede no enterarse nunca de que `flag` cambió.
- **Java Memory Model (JMM):**
 - Una **data race** son dos accesos a la misma variable, al menos uno escritura, sin sincronización que los ordene.
 - **DRF-SC:** si tu programa **no tiene data races**, se comporta como si fuera SC.
 - **volatile:** una escritura volatile *happens-before* toda lectura posterior de esa variable. Da **visibilidad** y **prohíbe reordenar**. **No da atomicidad:** `count++` con `volatile` sigue mal.
 - Ojo: `volatile int[]` a hace volátil la **referencia**, no los elementos → usar `AtomicIntegerArray`.
- **Idea para el parcial:** toda ejecución SC es también una ejecución válida en TSO/Java. Entonces, si algo **ya falla bajo SC**, sigue fallando en hardware real, y además pueden aparecer fallas nuevas.

6. Procesos vs threads (práctica 1, cultura general, poco probable en el parcial)

- **Proceso:** contenedor de recursos (espacio de memoria, archivos).
- **Thread:** lo que ejecuta. Tiene su propio stack, PC y registros, y **comparte la memoria** con los demás threads del proceso.
- **Concurrencia** (progreso intercalado, puede ser en 1 core) vs **paralelismo** (literalmente al mismo tiempo, en varios cores).
- **Modelos de mapeo** de threads de usuario a threads del kernel:
 - **N:1:** baratos, pero sin paralelismo.
 - **1:1:** los Thread clásicos de Java/pthreads.
 - **N:M:** los *virtual threads* de Java.

7. Guía 1 — qué entrena cada ejercicio

Ej	Concepto
1–4	semántica, diagramas de estados, contar trazas (baja prioridad)
5	carreras sobre una variable <code>found</code> (¿quién la pisa?)
6–8	razonar sobre interleavings: ¿qué salidas son posibles?, ¿puede no terminar?
9	bakery para 2 "simplificado", ¿resuelve mutex?
10	tickets con <code>PedirTurno</code> / <code>LiberarTurno</code> : mismo formato que el simulacro (a: no atómico, b: atómico)
11	Peterson generalizado a $n > 2$
12	<code>algunVerdadero</code> no atómico vs atómico
13	bakery sin el desempate <code>j < id</code>
14	fetch-and-add mal usado; hay que arreglarlo
15	<code>tomarFlag</code> atómico vs no atómico

Los ej 10–15 son el formato exacto del Ej 1 del parcial. Para cada uno: revisá las 3 propiedades y, para cada propiedad que falle, dá una traza.